

Manual Testing:

- Module 1: Testing concept (Theory), what?
- Module 2: Testing Project (Practical), How?
- Module 3: Agile process – Jira

Program: Instructions to the computer.

Module: Piece of functionality in application.

Version Control – software will be released in multiple versions.

Production environment: final stage of software development where the software is deployed for end-users to use.

Deploy: making software available for use on a device or server.

Software: software is a collection of computer programs which is help us to perform a task.

Types of software:

- **System Software:** Operating system, computer drivers, Servers, Utilities
- **Programming Software:** Computer Debuggers, Interpreter
- **Application Software:** mobile apps, desktop applications, Web Application

Software Testing: It is a part of software development process.

- Software testing is a activity of detecting and identifying the defects in the software.
- The objective of the tester is releasing quality product to the client.

Software Quality:

Parameters for which quality measures

- Bug free
- Delivered on time
- Meet requirements / expectations
- Within budget
- Maintain

Product vs Project:

If a software application is developed for specific customer based on the requirements, then it is called **project**.

If a software application is developed for multiple customers based on the market requirements, then it is called **product**.

Service Base Companies deals with project.

Ex: TCS, Accenture

Product Base Companies deals with the product.

Ex; Excel, Oracle

Error vs Bug vs Failure:

Error (Mistake) – A mistake made by a developer or tester while writing code or test cases.

Bug (Defect) – A defect (unexpected results) in the code that may or may not be detected.

Failure (Breakdown) – When a bug is triggered while using the application, causing it to malfunction. (When a bug is triggered and observed during testing)

Why the software has bugs?

- Miscommunication or no communication
- Software complexity
- Programming errors
- Changing requirements
- Lack of skilled tester

- Software development life cycle is a process used by software industries to design, develop and test the software.
- **SDLC** is a structured process that guides the development of high-quality software from **planning to deployment and maintenance**.
- It ensures that software is built efficiently, meets requirements, and is free from critical defects.
- **SDLC** is a step-by-step process that defines how software is planned, developed, tested, and deployed systematically to ensure high quality and efficiency.

Three P's – people, process, product

Phases:

Planning: Define the software's purpose and scope

Analysis: Gather and document user expectations

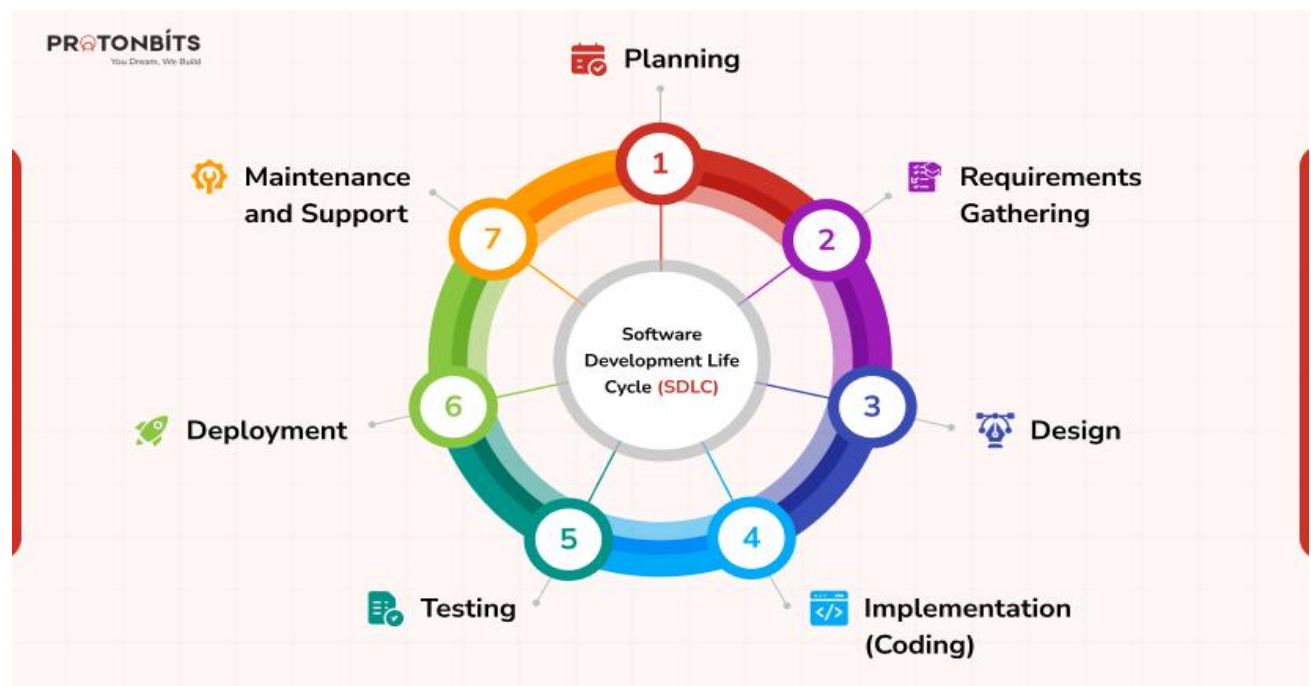
Design: Create the software's architecture and user interfaces

Development: Create the software

Testing: Test the software for quality assurance

Deployment: Deploy the software

Maintenance: Maintain the software



Waterfall model: (Old & traditional model)

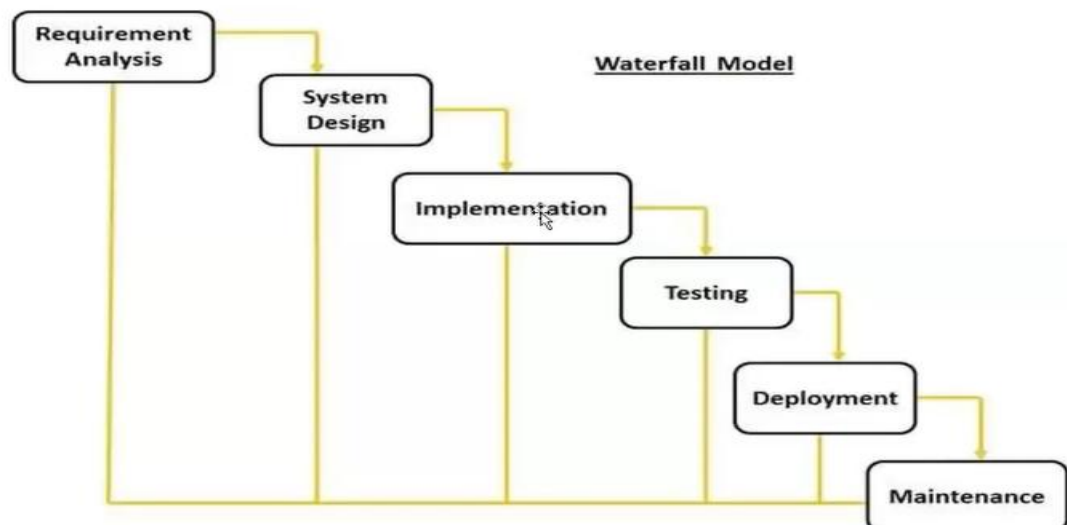
The **Waterfall Model** is a **linear** and **sequential** software development process where each phase must be completed before moving to the next. It has the following **phases**:

1. **Requirement Gathering & Analysis** – Collecting and documenting project requirements.
2. **System Design** – Creating architecture, UI design, and database structure.
3. **Implementation (Coding)** – Developers write code based on the design.
4. **Testing** – Finding and fixing bugs before deployment.
5. **Deployment** – Releasing the software to users. (Launching the website on a live server)
6. **Maintenance** – Fixing issues and updating the software as needed.

Key Characteristics of the Waterfall Model:

- ✓ **Sequential Process** – One phase starts only after the previous one is completed.
- ✓ **Easy to Manage** – Clear structure and documentation

Waterfall Model



Advantages:

- Quality of the product will be good
- Since recruitment changes are not allowed, chances of finding bugs will be less.
- initial investment is less since testers are hired at the later stages.
- Preferred for small project where requirements are freezed.

Disadvantage:

- Recruitment changes are not allowed
- if there is a defect in requirements phase that will be continued to later phases

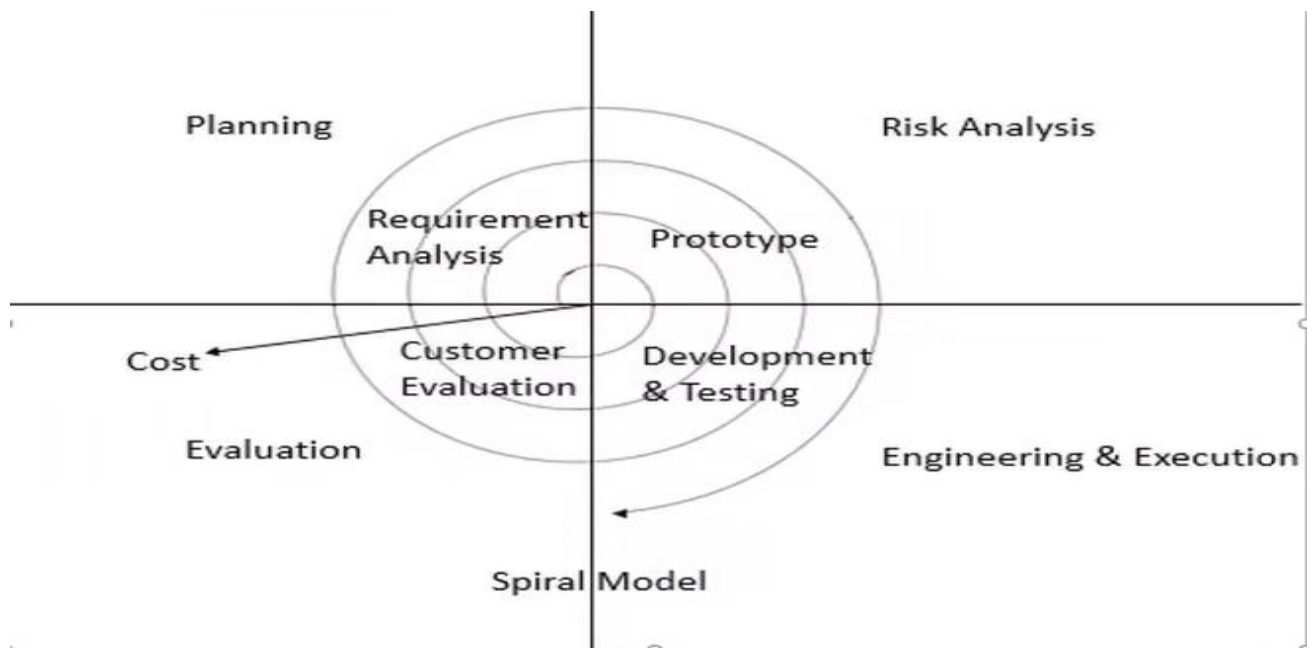
- total investment is more because time taking for rework on Defect is the consuming which leads to high investment.
- testing will be start after coding only the one testing is there.

Prototype model: (blueprint of the software) – btw waterfall & spiral model

- we get initial requirements and build the prototype, if customer satisfies then we will start design, coding, testing.

Spiral model:

- It focuses on **continuous refinement** through repeated cycles (spirals).
- It is also called iterative/version control model.
- To overcome the disadvantages of waterfall model.
- It is best for product base companies.



Phases of the Spiral Model:

Each spiral (iteration) consists of **four phases**:

1. **Planning** – Gathering requirements and identifying risks.
2. **Risk Analysis** – Evaluating potential risks and finding solutions.
 - Example: Identifying security vulnerabilities in payment processing.
3. **Development & Testing** – Coding, testing, and refining the product.
 - Example: Implementing and testing the login module before adding new features.

4. **Review & Refinement** – Gathering feedback, improving, and moving to the next iteration.

- Example: Users report UI issues, so the design is updated in the next spiral.

Key Characteristics of the Spiral Model:

✓ **Risk-Focused** – Identifies and manages risks at every phase.

✓ **Iterative** – Software is developed in repeated cycles, improving each time.

✓ **Flexible** – New changes can be made after each iteration.

✓ **Best for Large & Complex Projects** – Works well when requirements evolve.

Advantages:

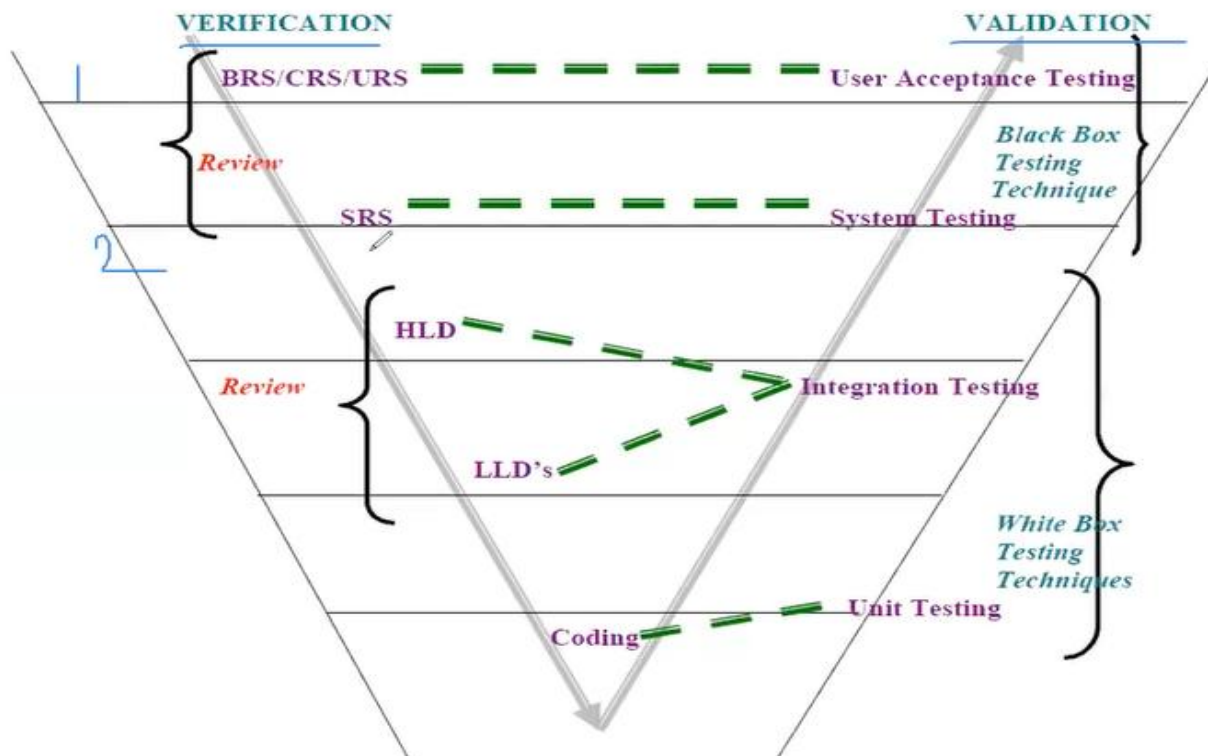
- Testing is done in every cycle, before going to next cycle.
- Customer will get to use the software for every module.
- Requirements changes are allowed after every cycle.

Disadvantages:

- Requirements changes are **NOT** allowed between the cycle.
- There is no testing in requirements & design phase.

V model:

- It also called vv model (verification & validation)
- It looks like V-shape
- In each phase, testing will conduct.



BRS/CRS/URS:

- Business unit prepare this document.
- Interact with customer to gather requirements.
- Understandable by business people not technical.
- In this phase user acceptance testing is done by business unit itself as through review, walkthrough or inception

Software Requirement Specifications (SRS) – it is a document in requirement phase.

- Project manager prepare this document.
- It is understandable by technical people.
- On the basis of SRS, designers will prepare HLD & LLD documents.

Verification & validation:

Verification:

- It checks whether we are building the right product.
- Focus on documentation.
- Verification typically involves:
Static testing techniques
- Testing the project related documentation is called **static testing techniques**.

1. Review:
2. Walkthrough:
3. Inception:

Validation:

- It checks whether we are building the product right.
- Focus on actual software.
- Takes place after verification done.
- Validation typically involves:
Dynamic Testing Techniques
- Testing the actual software is called **Dynamic Testing Techniques**.
 - Unit Testing
 - Integration Testing
 - System Testing
 - UAT Testing

Advantages:

- Testing is involved in each and every phase.

Disadvantages:

- Documentation is more.
- Initial Investment is more.

Review:

- Read the entire document and check whether the content is right or not.
- Review is conducted on documents to ensure correctness and completeness.

Types of Reviews:

1. Requirement review
2. Design review
3. Code review
4. Test plan review
5. Test case review etc.

Walkthrough:

- It's an informal review.
- We don't have any specific plan/meeting, it can be done whenever required.
- Author reads the document/code and discusses with peers.

Inspection:

- It's a most formal & preplanned review type.
- In which at least 3-8 people will sit in the meeting
 1. Reader – Author of the documentation.
 2. Writer – write down all issues/classification raised.
 3. Moderator – may be the organiser.
- Inspection will have a proper schedule which will be intimated via email to the concerned developer/tester.

QA vs QC vs QE:

Quality Assurance (QA)	Quality Control (QC)	Quality Engineer (QE)
It is process related.	It is the actual testing of the software.	Write the code to test the software.
They focus on building in quality.	Focuses on testing for quality.	
QA is preventing the defect.	QC is detecting the defect.	
Process oriented.	Product oriented.	
QA for entire lifecycle.	QC for testing part in sdlc.	

White box testing:

- Testing the application internal code and structure.
- Involves testing the application with detailed knowledge of its source code, architecture and configuration.

Black box testing:

- it focuses on the application's functionality from the end user's perspective.
- Analyses the application without knowing its internal design.
- Checks the system is working as expected and meets its requirements.

Levels of testing: (Stages of testing)

Unit Testing:

- Testing conduct on single module is called unit testing, then they integrate the all modules and perform integration testing.
- Both testing is conducted by developer itself for that we should have coding knowledge.
- Unit & Integration testing comes under white box testing techniques.
- Unit testing techniques
 1. Basis path testing (each line executed or not?)
 2. Control structural testing
 - a. Conditional coverage
 - b. Loop coverage
 3. Mutation testing

Integration Testing:

- Combine the multiple units as a single unit & then we check the data flow/communication btw the modules.
- Types of Integration testing:
 - Incremental testing
 - Incrementally adding the module & testing the data flow btw the modules.
 - One after another module.
 - Approaches:
 - Top down
 - Incrementally adding the module & testing the data flow btw the modules, and ensure the module added is the child of previous module.
 - Bottom up
 - Incrementally adding the module & testing the data flow btw the modules, and ensure the module added is the parent of previous module.
 - Sandwich/hybrid approach:
 - Combination of both approach is called sandwich approach.
 - Non-incremental integration testing:
 - Adding all the modules in a single shot and test the data flow btw modules.
 - Drawbacks:
 - It might miss data flow btw some of the modules.
 - If you find any defect, we can't understand the root cause of defect.

System Testing:

- Testing the overall functionality of the software whether it is working as client requirements or not.
- After completing this two testing's software is ready & deliver to the testing team to perform **system testing**.
 - They check whether the software working as per customer requirements or not.
- It focuses on below aspects:
 1. User interface testing (GUI)
 2. Functional testing
 3. Non-Functional testing
 4. Usability testing

UAT:

- User Acceptance Testing (UAT) done by customer along with tester.
- It comes under Black box testing techniques.
- UAT has two levels
 - Alpha testing: testing conducted in the company.
 - Beta testing: testing conducted in client environment.

System Testing:

User interface testing (GUI):

- Testing the user interface of an application.
- A graphical user interface includes all the elements such as fonts, colours, menus, checkbox, buttons, sizes, icons, content and images.

GUI Testing Checklist

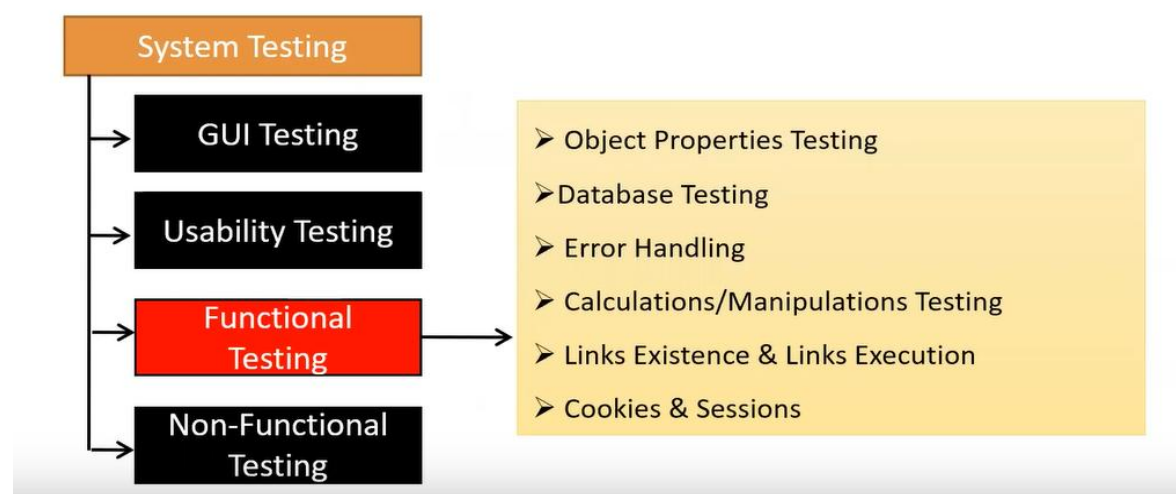
- Testing the size, position, width, height of the elements.
- Testing of the error messages that are getting displayed.
- Testing the different sections of the screen.
- Testing of the font whether it is readable or not.
- Testing of the screen in different resolutions with the help of zooming in and zooming out.
- Testing the alignment of the texts and other elements like icons, buttons, etc. are in proper place or not.
- Testing the colors of the fonts.
- Testing whether the image has good clarity or not.
- Testing the alignment of the images.
- Testing of the spelling.
- The user must not get frustrated while using the system interface.
- Testing whether the interface is attractive or not.
- Testing of the scrollbars according to the size of the page if any.
- Testing of the disabled fields if any.
- Testing of the size of the images.
- Testing of the headings whether it is properly aligned or not.
- Testing of the color of the hyperlink.
- Testing UI Elements like button, textbox, text area, check box, radio buttons, drop downs ,links etc.

Usability Testing:

- During this testing, validates application provides context sensitive help or not to the user.
- Check how easily the end user are able to understand and operate the application. (simply user friendly or not?)

Functional testing:

- Functionality means behaviour of the application.
- It talks about how your features should work.
- Focus on client requirements.
- Types of functional testing:

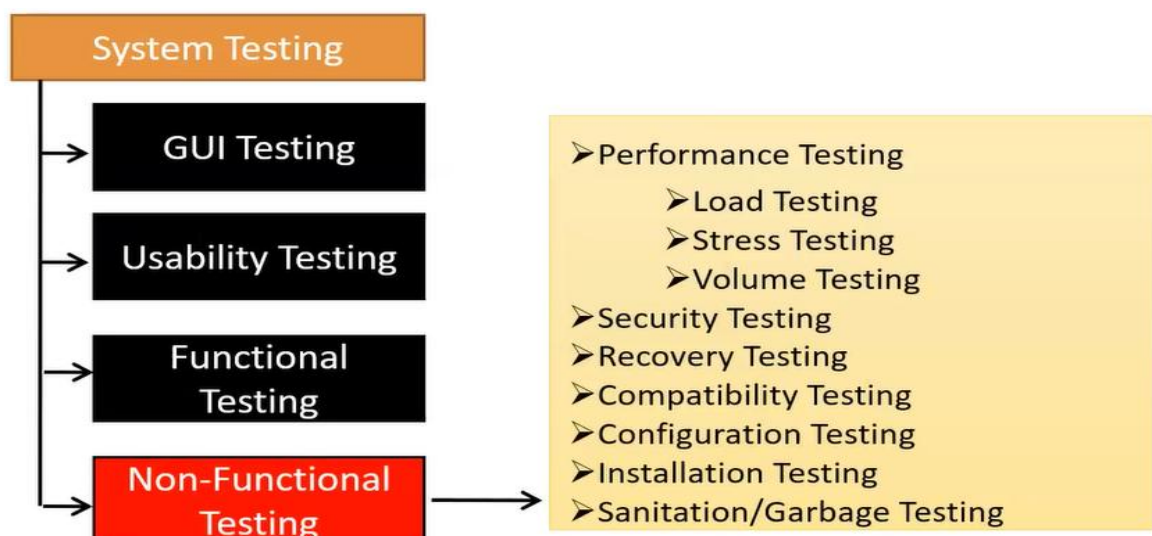


- Object properties testing:
 - Verify the properties of the object present on the app.
Ex; enable, disable, visible, focus.
- Database Testing:
 - It is also called backend testing.
 - Focus on DML (insert, delete, update, select)
 - Relation between the tables (Normalization).
 - **Normalization** is the process of organizing a database to reduce **redundancy** (duplicate data) and improve **data integrity**. It involves breaking down large tables into smaller, related tables while maintaining relationships.
 - Grey box testing (At UI level we perform white box and at backend black box)
 - Checking database operation w.r.t user operations.
- Error Handling Testing:
 - Verifying the error message while performing incorrect actions on the application.
 - Error message should be readable in simple language.

- Calculations & manipulation testing:
 - Tester should check the calculations.
- Link Existence & execution:
 - Where exactly the link is placed?
 - Links are navigating to proper page are not?
 - Types
 - Internal links: navigate to internal page of application.
 - External links: navigate to external applications.
 - Broken links: They don't have any pages.
- Cookies:
 - Temporary files which are created by the browser while browsing the page through internet.
 - These are created at client side.
- Session:
 - These are created at server side.
 - These are the time slot created by the server; session will be expired after some time. (if you idle for some time)

Non-Functionality Testing:

- Once app. functionality is stable then we do non-functionality testing.
- Focus on client expectations.
- Testing focus on performance, load, security etc.
- It can be done by special testers team.



- Performance testing: (speed of the application)

Load Testing	Stress Testing	Volume Testing
--------------	----------------	----------------

Gradually increasing the load on the application.	Suddenly increasing/decreasing the load on the application.	Check how system handles a large amount of data.
Check any chance to break the app. And check speed of the app.	“	“

- Security Testing:
 - Check how secure our application

Authentication	Authorization/Access Control
Verify the user is valid or not?	Check permissions of the valid user.
	What are the features he/she can access?

- Recovery Testing:
 - Check the system change to abnormal to normal.
- Compatibility Testing:
 - Check whether our software is compatible with different environment or not.
 - Three types of compatibilities are there:
 - Forward: Upgraded one is compatible with the new/future versions.
 - Backward: Upgraded software is compatible with the old versions.
 - Hardware Compatibility: (Configuration testing): **Configuration Testing** is a type of software testing that checks how an application performs under different **hardware, software, and network environments**. It ensures compatibility across various system configurations.
- Installation Testing:
 - Check the installation process for screen is clear to understand or not.
 - Simple or not.
 - Un-installation is easy and it delete all related documents.
- Sanitation/Garbage Testing:
 - If an application provides any extra feature/functionality then we consider it as bug.

Functional Testing Vs Non-Functional Testing

Functional Testing

- Validates functionality of Software.
- Functionality describes what software does.
- Concentrates on user requirement.
- Functional testing takes place before Non-functional testing.

Non-functional Testing

- Verify the performance, security, reliability of the software.
- Non-Functionality describes how software works.
- Concentrates on user expectation.
- Non-Functional testing performed after finishing Functional testing.

Software Testing Terminology

Regression Testing:

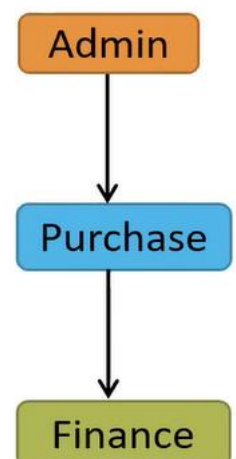
- Testing conducts on modified build to make sure there will not be impact on existing functionality because of changes like adding/deleting/modifying features.
- **Unit regression testing**
 - Testing only the changes/modifications done by the developer.
- **Regional Regression Testing**
 - Testing the modified module along with the impacted modules
 - Impact Analysis meeting conducts to identify impacted modules with QA & Dev.
- **Full Regression**
 - Testing the main feature & remaining part of the application.
 - Ex: Dev has done changes in many modules, instead of identifying impacted modules, we perform one round of full regression.

Re-Testing:

- Whenever the developer fixed a bug, tester will test the bug fix is called Re-testing.
- Tester close the bug if it worked otherwise re-open and send to developer.
- To ensure that the defects which were found and posted in the earlier build were fixed or not in the current build.
- Example
 - Build 1.0 was released. Test team found some defects (Defect Id 1.0.1, 1.0.2) and posted.
 - Build 1.1 was released, now testing the defects 1.0.1 and 1.0.2 in this build is retesting.

Difference between Regression and Retesting with Examples:

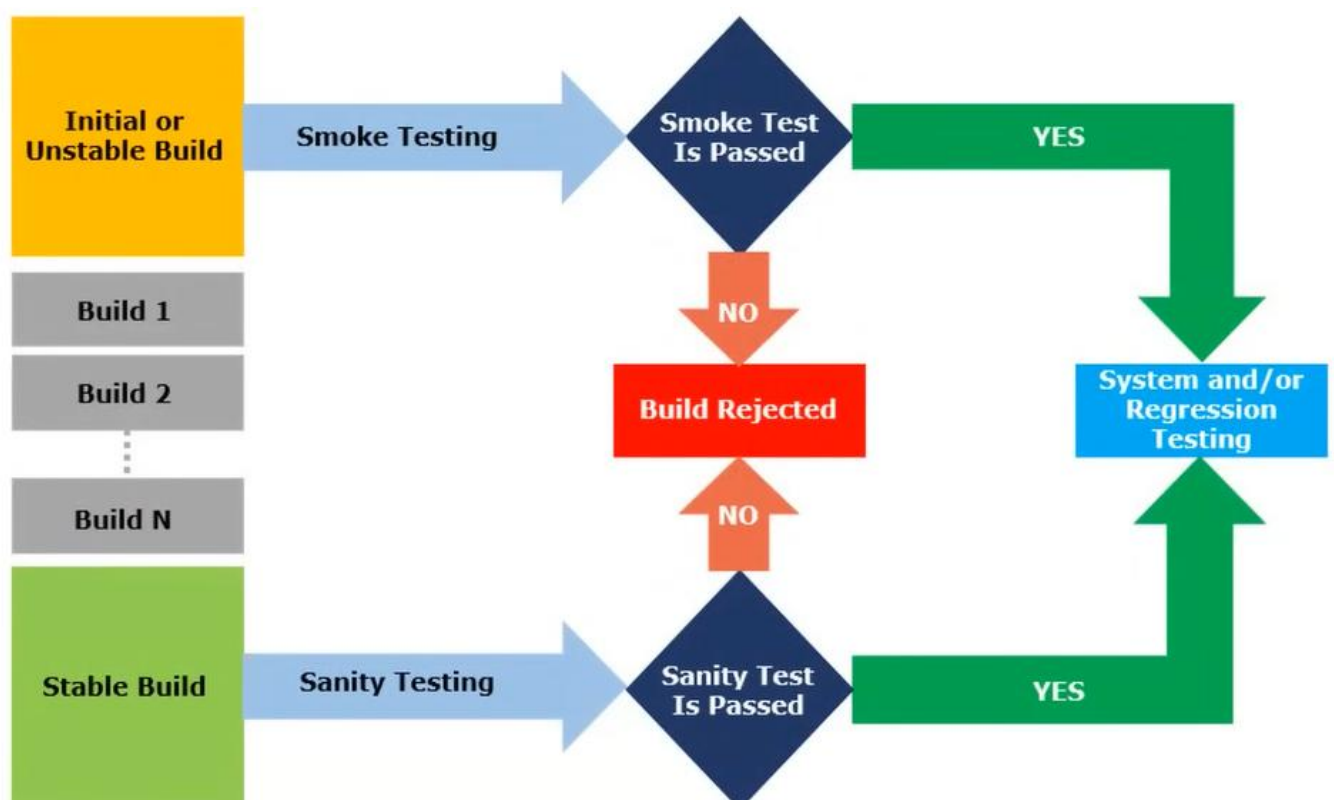
- An Application Under Test has three modules namely **Admin**, **Purchase** and **Finance**.
- Finance module depends on Purchase module.
- If a tester found a bug on Purchase module and posted. Once the bug is fixed, the tester needs to do **Retesting** to verify whether the bug related to the Purchase is fixed or not and also tester needs to do **Regression Testing** to test the Finance module which depends on the Purchase module.



Smoke and Sanity Testing:

- Setup env. & build develop during this process, if we face any issues, at the time of installation some files are missing or warning signs or errors immediately we reject the build.
- Smoke and Sanity Testing come into the picture after build release.

Smoke Testing	Sanity Testing
Smoke Test is done to make sure the build we received from the development team is testable/stable or not	Sanity Test is done during the release phase to check for the main functionalities of the application without going deeper.
Smoke Testing is performed by both Developers and Testers	Sanity Testing is performed by Testers alone
Smoke Testing, build may be either stable or unstable	Sanity Testing, build is relatively stable
It is done on initial builds.	It is done on stable builds.
It is a part of basic testing.	It is a part of regression testing.
Usually it is done every time there is a new build release.	It is planned when there is no enough time to do in-depth testing.



Exploratory Testing:

- We have to explore the application ,understand completely and test it.
- Understand the application , identify all possible scenarios , document it then use it for testing.
- We do exploratory testing when the Application ready but there is no requirement.
- Test Engineer will do exploratory testing when there is no requirement.
- **Drawbacks:**
- You might misunderstand any feature as a bug (or) any bug as a feature since you do not have requirement.
- Time consuming
- If there is any bug in application , you will never know about it.

Ad-hoc Testing:

- While executing the existing test cases we found some other bugs from corner.
- Testing application randomly without any test cases or any business requirement document.
- Adhoc testing is an informal testing type with an aim to break the system.
- Tester should have knowledge of application even thou he doesn't have requirements/test cases.
- This testing is usually an unplanned activity.
- No Documentation, No Test Design, No Test Case

Monkey/Gorilla Testing:

- Testing application randomly without any test cases or any business requirement document.
- Adhoc testing is an informal testing type with an aim to break the system.
- Tester do not have knowledge of application
- Suitable for gaming applications.

Ad hoc Testing Vs Monkey Testing Vs Exploratory Testing:

Adhoc Testing	Monkey Testing	Exploratory Testing
No Documentation	No Documentation	No Documentation
No Plan	No Plan	No Plan
Informal testing	Informal testing	Informal testing
Tester should know Application functionality	Testers doesn't know Application functionality	Testers doesn't know Application functionality
Random Testing	Random Testing	Random Testing
Intension is to break the application/find out corner defects	Intension is to break the application/find out corner defects	Intension is to learn or explore functionality of application
Any Applications	Gaming Applications	Any Applications which is new to tester

Positive Testing:

- Testing the application with **valid inputs** is called as Positive Testing.
- It checks whether an application behaves as expected with positive inputs.
- For example -

Enter Only Numbers

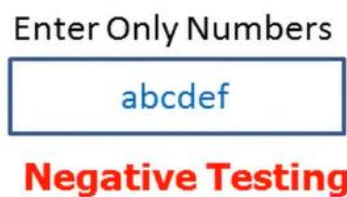
Positive Testing

There is a text box in an application which can accept only numbers. Entering values up to **99999** will be acceptable by the system and any other values apart from this should not be acceptable.

To do positive testing, set the valid input values from 0 to 99999 and check whether the system is accepting the values.

Negative Testing:

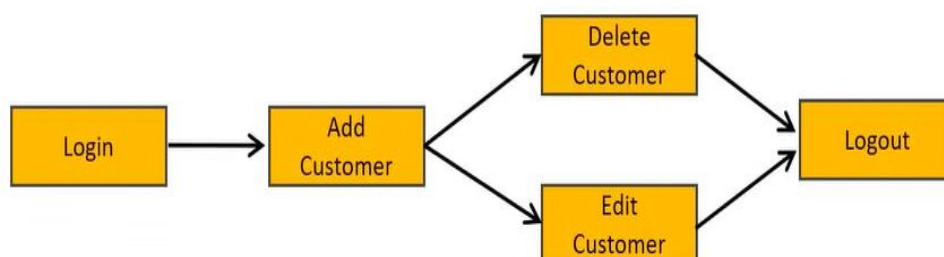
- Testing the application with **invalid inputs** is called as Negative Testing.
- It checks whether an application behaves as expected with the negative inputs.
- For example -



Negative testing can be performed by entering characters A to Z or from a to z. Either software system should not accept the values or else it should throw an error message for these invalid data inputs.

END-TO-END Testing:

- Testing the overall functionalities of the system including the data integration among all the modules is called end-to-end testing.



End-To-End Test

- 1) Login
- 2) ADD New Customer
- 3) Edit customer
- 4) Delete Customer
- 5) Logout

Globalization and Localization Testing:

- Globalization Testing is called Internationalization/I18 Testing

- **Globalization Testing:**

- Performed to ensure the system or software application can run in **any cultural or local environment**.
- Different aspects of the software application are tested to ensure that it supports every language and different attributes.
- It tests the different currency formats, mobile number formats and address formats are supported by the application.
- For example, Facebook.com supports many of the languages and it can be accessed by people of different countries. Hence it is a globalized product.

- **Localization Testing:**

- Performed to check system or software application for a **specific geographical and cultural environment**.
- Localized product only supports the specific kind of language and is usable only in specific region.
- It tests the specific currency format, mobile number format and address format is working properly or not.
- For example, Baidu.com supports only the Chinese language and can be accessed only by people of few countries. Hence it is a localized product.

Test Design Techniques:

- It's also called Test Design Techniques/Test Data Design Techniques/Test case Design Techniques
- These techniques are used to prepare data for testing.
 - 1) Reduce the Data
 - 2) More Coverage: Minimum data that cover all the functionalities.
- Types:
 1. Equivalence Class Partitioning (ECP)
 2. Boundary Value Analysis (BVA)

3. Decision Table based testing.
4. State Transition
5. Error Guessing

Equivalence Class partitioning (ECP):

- classify/divide/partition the data in to multiple classes.
- Partition data into various classes and we can select data according to class then test. It reduce the number of test-cases and saves time for testing.

Enter a Number: * Allow Digits from 1--500

Normal Test Data

1
2
3
4
.
.
.
.
500

Divide values into Equivalence Classes

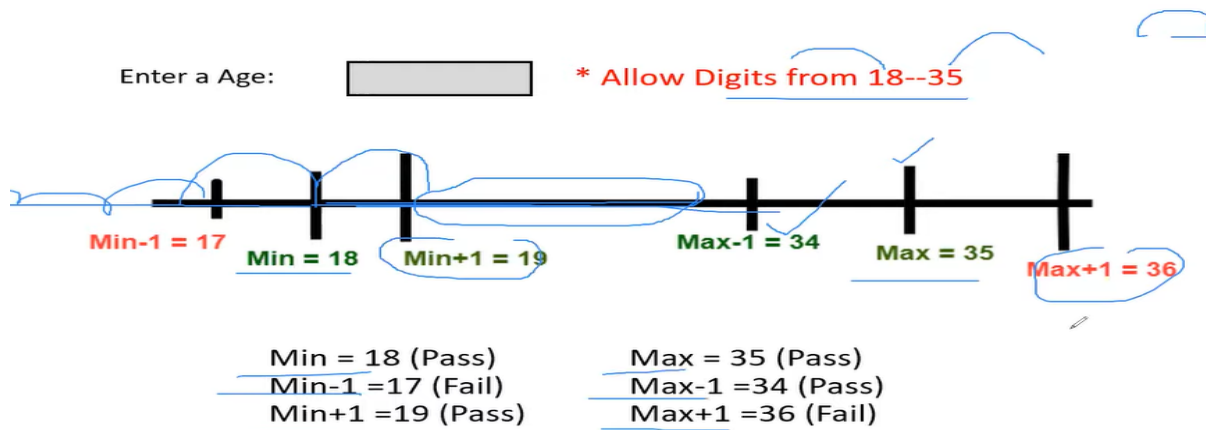
-100 to 0 → -50 (Invalid)
1 – 100 → 30 (Valid)
101 – 200 → 160 (Valid)
201 – 300 → 250 (Valid)
301 – 400 → 320 (Valid)
401 – 500 → 450 (Valid)
501 – 600 → 550 (Invalid)

Test Data using ECP

-50
30
160
250
320
450
550

Boundary value Analysis (BVA):

- BVA technique used to check Boundaries of the input.



Input domain Testing: (Testing type)

- The value will be verified in the text box/input fields.
- We use ECP & BVA techniques in input domain testing.

Decision Table:

- Decision Table is also called as Cause-Effect Table.
- This technique will be used if we have more conditions and corresponding actions.
- In Decision table technique, we deal with combinations of inputs.
- To identify the test cases with decision table, we consider conditions and actions.

Example:

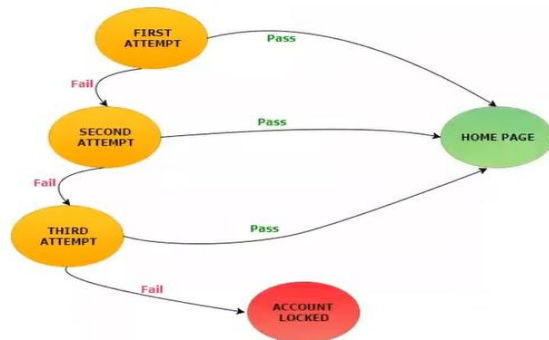
- Take an example of transferring money online to an account which is already added and approved.
- Here the **conditions** to transfer money are
 - Account already approved
 - OTP (one time password) matched
 - Sufficient money in the account
- And the **actions** performed are
 - Transfer money
 - Show a message as insufficient amount
 - Block the transaction in case of suspicious transaction

		TC1	TC2	TC3	TC4	TC5
Condition1	Account already approved	TRUE	TRUE	TRUE	TRUE	FALSE
Condition2	OTP Matched	TRUE	TRUE	FALSE	FALSE	X
Condition3	Sufficient Money in the Account	TRUE	FALSE	TRUE	FALSE	X
Action1	Transfer Money	Execute				
Action2	Show message 'Insufficient Amount'		Execute			
Action3	Block the transaction in case of Suspicious Transaction			Execute	Execute	X

State Transition:

- State transition means changing from one position to another.
- In State Transition technique changes in input conditions change the state of the Application.
- This testing technique allows the tester to test the behavior of an AUT.
- The tester can perform this action by entering various input conditions in a sequence.
- In State transition technique, the testing team provides positive as well as negative input test values for evaluating the system behavior.

- Take an example of login page of an application which locks the user name after three wrong attempts of password.



STATE	LOGIN	CORRENT PASSWORD	INCORRECT PASSWORD
S1	First Attempt	S4	S2
S2	Second Attempt	S4	S3
S3	Third Attempt	S4	S5
S4	Home Page		
S5	Display a message as "Account Locked, please consult Administrator"		

Error Guessing:

- Its also known as data management technique.
- Error guessing is one of the testing techniques used to find bugs in a software application based on tester's prior experience.
- In Error guessing we don't follow any specific rules.
- It depends on Tester Analytical skills and experience.
- Some of the examples are:
 - Submitting a form without entering values.
 - Entering invalid values such as entering alphabets in the numeric field.

STLC:

- Software Testing Life Cycle (STLC) is a step-by-step process that defines how testing should be carried out systematically to ensure software quality. It consists of six major phases:

Phases of STLC:

1. **Requirement Analysis** – Understanding what needs to be tested.
 - ♦ Example: Reviewing the software requirements to identify testable functionalities.
2. **Test Planning** – Defining strategy, scope, timeline, and resources.
 - ♦ Example: Deciding which types of testing (functional, security, performance) are needed.
3. **Test Case Design & Development** – Writing test cases and preparing test data.

- ◆ Example: Creating test cases for login functionality (valid & invalid inputs).
- 4. **Test Environment Setup** – Preparing the test environment (servers, databases, tools).
 - ◆ Example: Setting up a staging server that mimics the production environment.
- 5. **Test Execution** – Running test cases and reporting defects.
 - ◆ Example: Executing test cases on a web application and logging bugs in a bug-tracking tool.
- 6. **Test Closure** – Evaluating testing, preparing reports, and improving future testing.
 - ◆ Example: Reviewing defects found, summarizing test coverage, and suggesting improvements.



SNO	PHASE	Input	Activities	Responsibility	Out Come
1	Test Planning	Project Plan	➤Identify the Resources	Test Lead/Team Lead (70%)	Test Plan Document
	What to test	Functional Requirements	➤Team Formation	Test Manager (30%)	
	How to test		➤Test Estimation		
	when to test		➤Preparation of Test Plan		
			➤Reviews on Test Plan		
			➤Test Plan Sign-off		
2	Test Designing	Project Plan	➤Preparation of Test Scenarios	Test Lead/Team Lead(30%)	Test Cases Document
		Functional Requirements	➤Preparation of Test Cases	Test Engineers(70%)	Traceability Matrix
		Test Plan	➤Reviews on Test Cases		
		Design Docs	➤Traceability Matrix		
		Use cases	➤Test Cases Sign-off		
3	Test Execution	Functional Requirements	➤Executing Test cases	Test Lead/Team Lead(10%)	Status/Test Reports
		Test Plan	➤Preparation of Test Report/Test Log	Test Engineers (90%)	
		Test Cases	➤Identifying Defects		
		Build from Development Team			
4	Defect Reporting & Tracking	Test Cases	➤Preparation of Defect Report	Test Lead/Team Lead(10%)	Defect Report
		Test Reports/Test Log	➤Reporting Defects to Developers	Test Engineers (90%)	
5	Test Closure/Sign-Off	Test Reports	➤Analyzing Test Reports	Test Lead/Test Manger(70%)	Test Summary Reports
		Defect Reports	➤Analyzing Bug Reporting	Test Enginners(30%)	
			➤Evaluating Exit Criteria		

Why is STLC Important?

- ✓ Ensures structured testing for **better defect detection**.
- ✓ Reduces risks by identifying **bugs early**.
- ✓ Improves **software quality and reliability**.

Test Planning:

- Test plan is a document says about scope of the testing like which features to be tested, schedules and tools, who will conduct& when, testing types.
- It's prepared by project/product manager. (higher well management)
- It includes timelines and they identify the resources.
- The final outcome is Test Plan Document.

Test Plan Contents

- A Test Plan is a document that describes the test scope, test strategy, objectives, schedule, deliverables and resources required to perform testing for a software product.
- **Test plan template contents:**
 - Overview
 - Scope
 - Inclusions
 - Test Environments
 - Exclusions
 - Test Strategy
 - Defect Reporting Procedure
 - Roles/Responsibilities
 - Test Schedule
 - Test Deliverables
 - Pricing
 - Entry and Exit Criteria
 - Suspension and Resumption Criteria
 - Tools
 - Risks and Mitigations
 - Approvals

Terminology:

Use Case V/s Test Case

- **Use Case** – Describes functional requirement, prepared by Business Analyst(BA).
- **Test Case** – Describes Test Steps/ Procedure, prepared by Test Engineer.

Use case, Test Scenario & Test Case

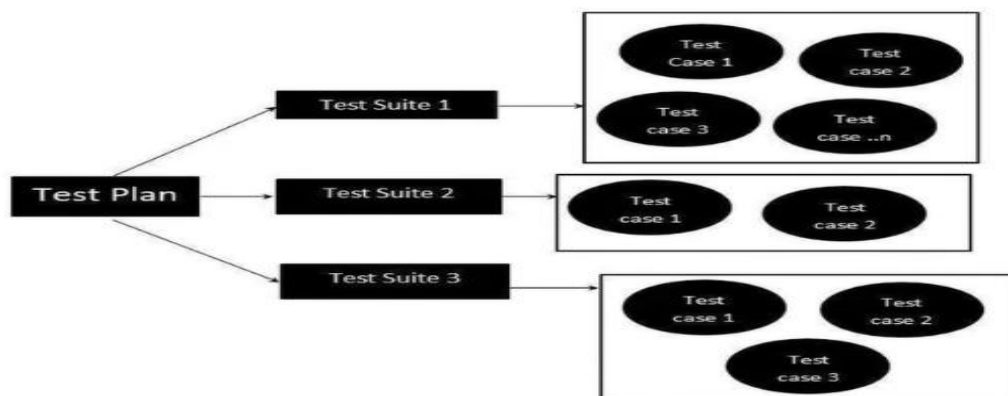
- **Use Case:**
 - Use case describes the requirement.
 - Use case contains THREE Items.
 - **Actor**, which is the user, which can be a single person or a group of people, interacting with a process.
 - **Action**, which is to reach the final outcome
 - **Goal/Outcome**, which is the successful user outcome.
- **Test Scenario:**
 - A possible area to be tested (What to test)
- **Test Case:**
 - Step by step actions to be performed to validate functionality of AUT (How to test).
 - Test case contains test steps, expected result & actual result.

Test Scenario V/s Test Case

- Test Scenario is '**What to be tested**' and Test Case is '**How to be tested**'.
- **Example:-**
- Test Scenario: Checking the functionality of Login button
 - TC1: Click the button without entering user name and password.
 - TC2: Click the button only entering User name.
 - TC3: Click the button while entering wrong user name and wrong password.

Test Suite

- **Test Suite** is group of test cases which belongs to same category.



Test Case:

- A Test Case is a set of actions executed to validate particular feature or functionality of your software application.
- In tech design we prepare test case document.
- Test case Document Contents:

- Test Case ID
 - Test Case Title
 - Description
 - Pre-condition
 - Priority (P0, P1,P2,P3) – order
 - Requirement ID
 - Steps/Actions
 - Expected Result
 - Actual Result
 - Test data
- Test case Template (in excel)

	A	B	C	D	E	F	G	H	I	J	K	L	M
	Module Name	Req ID	TestType	Priority	Test Case ID	Test Scenario	Precondition	Testcases / Teststeps	Actual Result	Expected Result	Result		
1	Homepage	1	+ve	p1	1	Verify the URL of the home page	1. Open the browser 2. Enter the URL:http://spicejet.com/ Click on Go or Press Enter	1. Open the browser 2. Enter the URL http://spicejet.com/ 3. Click on Go or Press Enter		Browser should navigate to the home page			
2	BookaFlight	2	GUI	P3	1	Verify the GUI of home page	Open the URL http://spicejet.com	1. Check the spell of all the fields 2. Check alignment of all the fields 3. Check the font of all the fields 4. Check the color of all the fields 5. Check the look and feel of the page	Application is maintaining the consistency	Application should maintain the consistency			
3	BookaFlight	2	+ve	P1	2	Verify the fields in bookaflight page	Open the URL http://spicejet.com	Check the below fields in home page Field Name Field Type Round trip Radio Oneway Radio Leaving From Dropdown Going To Dropdown Date Picker1 Date Picker Date Picker2 Date Picker Adult Dropdown Children Dropdown Infants Dropdown Indian armed forces personnel Checkbox Student Check box Find flights Button	Student discount is displaying instead of student	Application should display all the fields in bookaflight page	Pass Fail		

Requirement Traceability Matrix:

- RTM describes the mapping of requirements with test cases.
- The main purpose of RTM is to see all the test cases are covered so that no functionality should miss while doing software testing.
- Requirement Traceability matrix – Parameters Include:
 - Req. ID
 - Req. Des.
 - Test Case ID

Test/QA Environment:

- It's a platform which is exactly the replica of customer environment.
- It specially builds for test case execution on the software production.
- It's created by integrating the required software and hardware along with proper network configurations.

- Test env. Simulates production/real time env.
- Another name of test environment is **Test Bed**.

Test Execution:

- During this phase test team will carry out the testing based on the test plans and the test cases prepared.
- **Entry Criteria:** Test cases , Test Data & Test Plan
- **Activities:**
 - Test cases are executed based on the test planning.
 - Status of test cases are marked, like Passed, Failed, Blocked, Run, and others.
 - Documentation of test results and log defects for failed cases is done.
 - All the blocked and failed test cases are assigned bug ids.
 - Retesting once the defects are fixed.
 - Defects are tracked till closure.
- **Deliverables:** Provides defect and test case execution report with completed results.

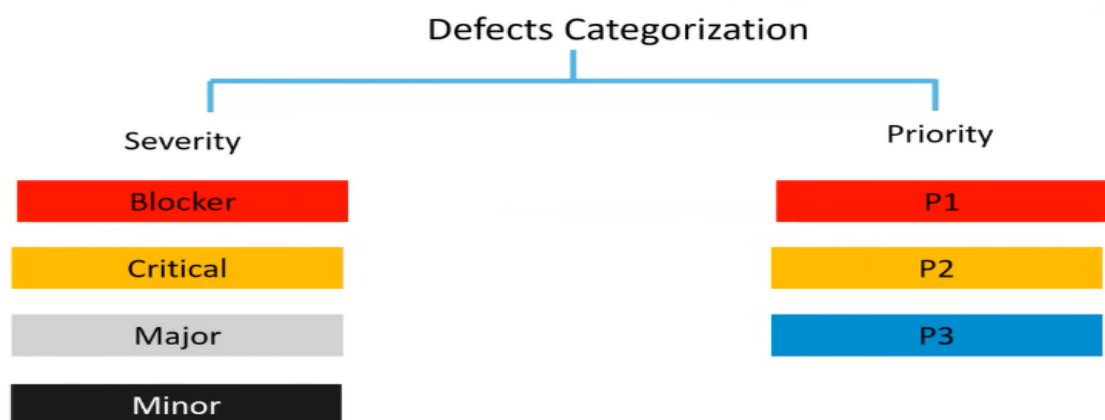


Guidelines for Test Execution

- The Build being deployed to the QA environment is the most important part of the test execution cycle.
- Test execution is done in Quality Assurance (QA) environment.
- Test execution happens in multiple cycles.
- Test execution phase consists Executing the test cases + test scripts(if automation).

Defect/Bugs:

- Any mismatched functionality found in an application is called as Defect/Bug/Issue.
- During Test Execution Test engineers are reporting mismatches as defects to developers through templates or using tools.
- Defect Reporting Tools:
 - Clear Quest
 - DevTrack
 - Jira
 - Quality Centre
 - Bug Jilla etc.
- Defect Report Contents:
 - **Defect ID** - Unique identification number for the defect.
 - **Defect Description** - Detailed description of the defect including information about the module in which defect was found.
 - **Version** - Version of the application in which defect was found.
 - **Steps** - Detailed steps along with screenshots with which the developer can reproduce the defects.
 - **Date Raised** - Date when the defect is raised
 - **Reference** - where you Provide reference to the documents like requirements, design, architecture or may be even screenshots of the error to help understand the defect.
 - **Detected By** - Name/ID of the tester who raised the defect
 - **Status** - Status of the defect, more on this later
 - **Fixed by** - Name/ID of the developer who fixed it
 - **Date Closed** - Date when the defect is closed
 - **Severity** which describes the impact of the defect on the application
 - **Priority** which is related to defect fixing urgency.
- Defect Categories:



Defect Severity:

- Severity describes the seriousness of defect and how much impact on Business workflow.
- **Defect severity can be categorized into four class**
 - **Blocker(Show stopper):** This defect indicates nothing can proceed further.
 - Ex: Application crashed, Login Not worked
 - **Critical :** The main/basic functionality is not working. Customer business workflow is broken. They cannot proceed further.
 - Ex1: Fund transfer is not working in net banking
 - Ex2: Ordering product in ecommerce application is not working.
 - **Major:** It cause some undesirable behavior, but the feature/application is still functional.
 - Ex1: After sending email there is no confirm message
 - Ex2: After booking cab there is no confirmation.
 - **Minor:** It won't cause any major break-down of the system
 - Ex: Look and feel issues, spellings, alignments.

Defect Priority:

- Priority describes the importance of defect.
- Defect Priority states the order in which a defect should be fixed.
- **Defect priority can be categorized into three class**
 - **P0 (High) :** The defect must be resolved immediately as it affects the system severely and cannot be used until it is fixed.
 - **P1 (Medium):** It can wait until a new versions/builds is created
 - **P2 (Low):** Developer can fix it in later releases.

		Priority	
		High	Low
Severity	High	Login is taking to the blank page.	About Us link is going to blank page.
	Low	After user is logged into application, he can see Home Page. But there is spelling mistake in <u>Home Page</u> .	User opened contact page. Email ID has spelling mistake.

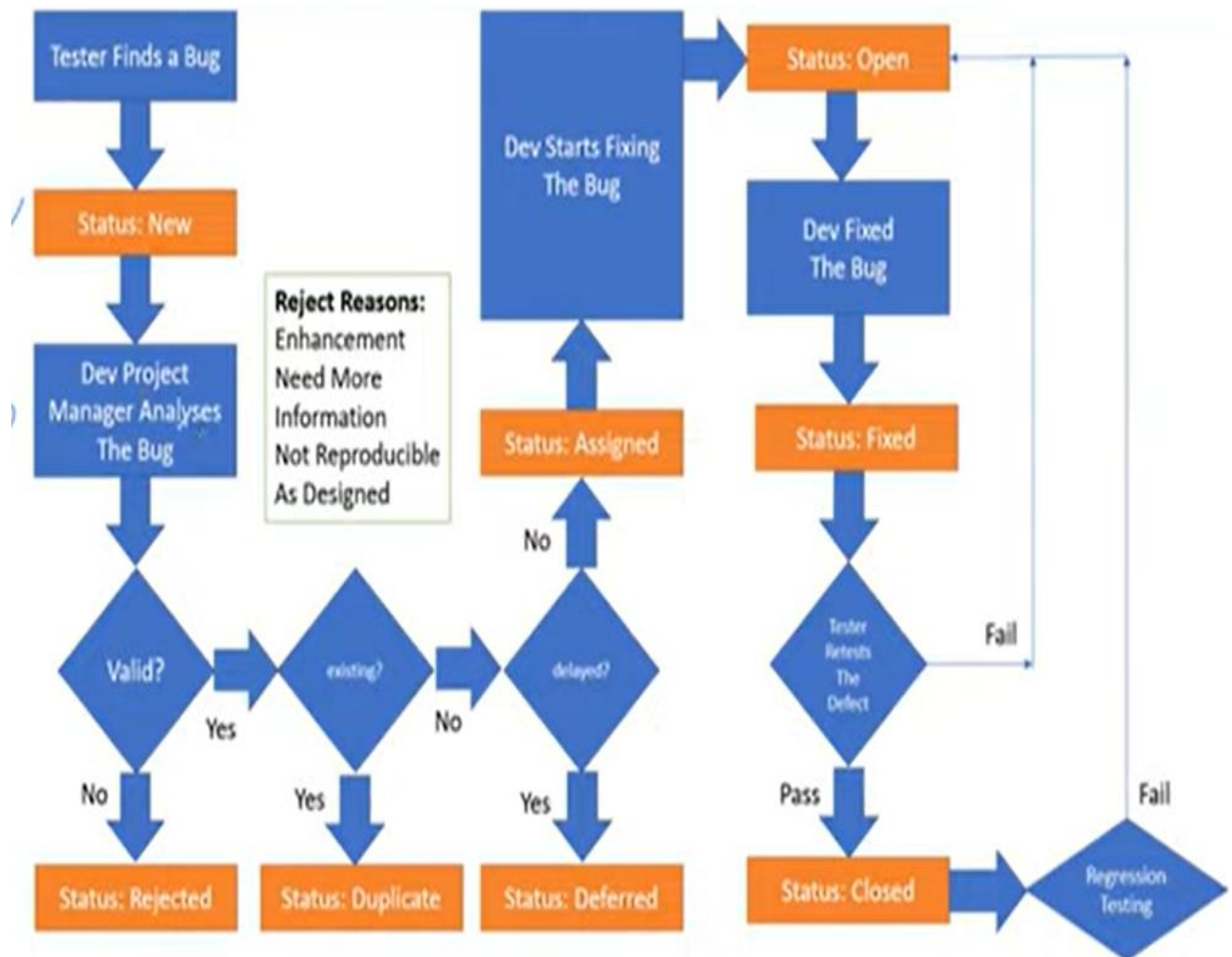
More examples...

- **Low priority-Low severity** - A spelling mistake in a page not frequently navigated by users.
- **Low priority-High severity** - Application crashing in some very corner case.
- **High priority-Low severity** - Slight change in logo color or spelling mistake in company name.
- **High priority-High severity** - Issue with login functionality.(user is not able to login to the application)
- **High Severity- Low Priority** - Web page not found when user clicks on a link (user does not visit that page generally)
- **Low Priority- Low Severity** - Any cosmetic or spelling issues which is within a paragraph or in the page

Defect Resolution:

- After receiving the defect report from the testing team, development team conduct a review meeting to fix defects. Then they send a Resolution Type to the testing team for further communication.
- Developer has some opinion on defect.
- Resolution Types:
 - Accept
 - Reject
 - Duplicate
 - Enhancement: It's not a defect maybe feature in coming year.
 - Need more information
 - Not Reproducible
 - Fixed
 - As Designed

Bug Life Cycle: (Deferred means bugs are in the queue)



Test Cycle Closure:

- Activities
 - Evaluate cycle completion criteria based on Time, Test coverage, Cost, Software, critical business objectives, quality.
 - Prepare test metrics based on the above parameters.
 - Document the learning out of the project.
 - Prepare Test summary report
 - Qualitative and quantitative reporting of quality of the work product to the customer.
 - Test result analysis to find out the defect distribution by type and severity.
- Deliverables:
 - Test Closure Report
 - Test Matrics

Test Matrices required the data to calculate its percentage:

SNO	Required Data
1	No. Of Requirements
2	Avg. No. of Test Cases written Per Requirement
3	Total No.of Test Cases written for all Requirement
4	Total No. Of test cases Executed
5	No.of Test Cases Passed
6	No.of Test Cases Failed
7	No.of Test cases Blocked
8	No. Of Test Cases Un Executed
9	Total No. Of Defects Identified
10	Critical Defects Count
11	Higher Defects Count
12	Medium Defects Count
13	Low Defects Count
14	Customer Defects
15	No.of defects found in UAT

- **% of Test cases Executed:**
 $\text{No.of Test cases executed} / \text{Total No. of Test cases written}) * 100$
- **% of test cases NOT executed:**
 $(\text{No.of Test cases NOT executed} / \text{Total No. of Test cases written}) * 100$
- **% Test cases passed**
 $(\text{No.of Test cases Passed} / \text{Total Test cases executed}) * 100$
- **% Test cases failed**
 $(\text{No.of Test cases failed} / \text{Total Test cases executed}) * 100$
- **%Test cases blocked**
 $(\text{No.of test cases blocked} / \text{Total Test cases executed}) * 100$
- **Defect Density: Number of defects identified per requirement/s**
 $\text{No.of defects found} / \text{Size(No. of requirements)}$
- **Defect Removal Efficiency (DRE):**
 $(A / A+B) * 100$
 $(\text{Fixed Defects} / (\text{Fixed Defects} + \text{Missed defects})) * 100$
 - A- Defects identified during testing/ Fixed Defects
 - B- Defects identified by the customer/Missed defects
- **Defect Leakage:**
 $(\text{No.of defects found in UAT} / \text{No. of defects found in Testing}) * 100$
- **Defect Rejection Ratio:**
 $(\text{No. of defect rejected} / \text{Total No. of defects raised}) * 100$
- **Defect Age:** Fixed date-Reported date
- **Customer satisfaction** = No.of complaints per Period of time

QA/Testing Activities:

- Understanding the requirements and functional specifications of the application.
- Identifying required Test Scenario's.
- Designing Test Cases to validate application.
- Setting up Test Environment (Test Bed)
- Execute Test Cases to valid application
- Log Test results (How many tests cases pass/fail).
- Defect reporting and tracking.
- Retest fixed defects of previous build
- Perform various types of testing in application.
- Reports to Test Lead about the status of assigned tasks
- Participated in regular team meetings.
- Creating automation scripts.
- Provides recommendation on whether or not the application/system is ready for production.

7 Principles of software Testing:

1. Start software testing at early stages. Means from the beginning when you get the requirements.
2. Test the software in order to find the defects.
3. Highly impossible to give the bug free software to the customer.
4. Should not do Exhaustive testing. Means we should not use same type of data for testing every time.
5. Testing is context based. Means decide what types of testing should be conducted based on type of application.
6. We should follow the concept of Pesticide Paradox. Means, if you are executing same cases for longer run, they won't be finding any defects. We have to keep update test cases in every cycle/release in order to find more defects.
7. We should follow defect clustering. Means some of the modules contains most of the defects. By experience, we can identify such risky modules. 80% of the problems are found in 20% of the modules.

Metrics: Track it/measure it

Exhaustive Testing: Use different types of data.

Context based: Based upon type of application.

Shift Left & Shift Right Testing Questions

1. What is shift right testing – Testing done after the software is released (e.g., monitoring, feedback, A/B testing).
2. Shift left testing example – Writing test cases during requirements gathering or unit testing early in development.
3. What is shift left approach in Agile – Testing starts early in the Agile sprint lifecycle.
4. How to implement shift left testing – Start testing during planning, involve QA in design, use unit & integration tests.
5. Benefits of shift left testing – Early bug detection, reduced cost, faster delivery.
6. Shift left testing pyramid – A model showing more tests at unit level, fewer at UI level (Unit > Integration > UI).
7. Shift left testing in Agile – Testing is done in parallel with development within sprints.
8. Shift-left testing tools – Tools like JUnit, Selenium, Jenkins, SonarQube.
9. What is shift left approach – Moving testing earlier in the development phase.
10. What is shift left testing in software testing – Performing test planning, unit

More Related Questions

13. Software testing situation-based interview questions – Questions where you explain how you'd react to real testing issues (e.g., production bug reported by a client).
 14. Software testing scenarios interview questions – How you'd handle scenarios like broken links, slow performance, or test failure.
 15. What is shift left approach in testing? – Involving testing activities from the early stages of the software lifecycle.
 16. What is the difference between TDD and shift left testing?
- TDD (Test Driven Development) – Writing tests before code.
- Shift Left – General approach of testing early, not just with TDD.

17. What is an example of a shift left? – Writing test cases before development starts or doing code review by QA.

18. Scenario-based software testing interview questions and answers for experienced PDF – A downloadable list of expert-level Q&A.

19. What is scenario-based testing? – Testing based on real user actions or workflows.

PROJECT

Manual Testing project in 7 Stages:

1. Project Introduction
2. Understanding & explore the functionality (FRS)
3. Test plan prepared by Test Lead along with Test Engineer.
4. Writing Test Scenarios.
5. Writing Test case & Review.
6. Environment Setup & build deployment.
7. Test Execution
8. Bug Reporting & Tracking
9. Sanity Testing, Retesting & Regression Testing
10. Test Sign-off

Website: Opencart.com

Mock up Screens: Dummy screens which will design before actual software is designed.

Entry criteria: List of documents tester needed.

Exit Criteria: List of req. explored by tester

Webserver is used to deploy the software.

XAMP:

- Change all file in source code from config-*.php to config.php

You can connect to the database through client application.

Cmd: use openshop;

Select * from oc_customer;

Review test scenario, test plan documents.

Agile Model / Methodology / process

(A ja)

- It's a iterative & incremental approach/process.
- Agile is a process with defined principles.
- It's an iterative process that involves breaking a project into phases and regularly delivering working software.
- The Agile process is a project management approach that emphasizes collaboration, flexibility, and continuous improvement.

Iterative: same kind of process we're repeating again & again.

Incremental: We'll implement some features at the beginning and then we keep adding new features/modules to the existing software.

Principles:

- Customer no need to wait for long time.
- We develop, test & release piece of software to the customer with few no. of features.
- We can accept/accommodate req. changes at anytime at the sd process.

Advantages:

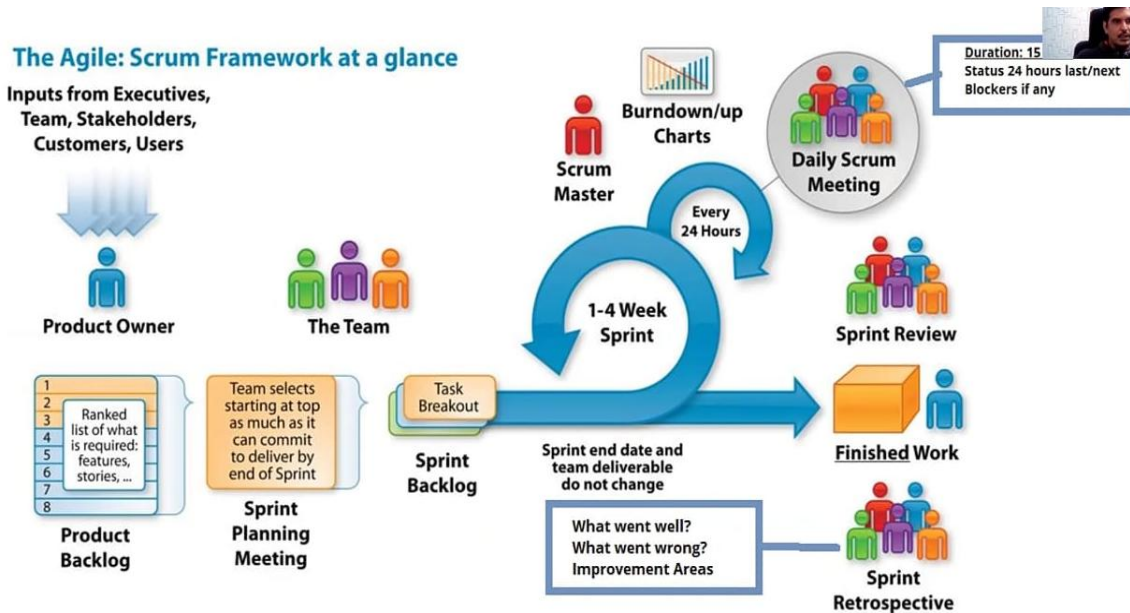
- Req. changes are allowed in any stage.
- Release will be very fast.
- Customer no need to wait for long time.
- Good comm. Btw team, they collectively work towards one goal (giving quality product)
- It's very easy model to adopt.

Disadvantages:

- Less focus on design & document.

Scrum:

- It's a framework (conceptual structure of building of something) through which we build software product by following agile principles.
- Scrum includes group of people called as scrum team. Normally contains 5-9 members.
 - o Product owner
 - o Scrum master
 - o Dev team
 - o QA Team



Product owner:

- Define the features of the product. (he'll also connect to the customer)
- Prioritize features according to market value.
- Adjust features & priority in every iteration as needed.
- Accept or reject work results.

Scrum Master:

- The main role is facilitating & driving the agile process.
- He's aware of entire agile process.

Scrum Terminology:

User story:

- A feature/module in a software.
- A simple description of a product feature perspective from end-user.

Epic: Collection of user stories.

Product Backlog:

- Contains list of user stories.
- Prepared by product owner.(it's a document)

Sprint/iterative:

- Period of time/span to complete the user stories, decided by the product owner and team usually 2-4 weeks of time.
- Complete design, development, test for particular features.

Sprint planning meeting:

- Meeting conducted with the team to define what can be delivered in the sprint duration.
- How many stories we have in backlog, develop and test during the sprint & duration of the sprint. (1-day meeting)

Sprint backlog:

- List of committed stories by dev/QA for specific sprint.

Scrum meeting:

- Meeting conducted by scrum master everyday 15 mins called as standup meeting/scrums call.
 - o Any particular feature/req. missing.
 - o Dev/Test not able to do something?
 - o Scrum master track the entire status in the agile process.
 - o Task planned yesterday/today/tomorrow
 - o If any delay/blocker while delivering software

(clearly discuss in meeting)

Sprint retrospective meeting:

- Conduct meeting only 1 time after completion of sprint. The entire team will participate.
- What went wrong in previous sprint, any other implementations in upcoming sprint.
-

Story point:

- Rough estimation of user stories, will be given by dev & QA in the form of Fibonacci series.

Fibonacci series: The sequence number starts with 0 & 1 after that each number is sum of two numbers before it.

Burndown charts:

- Shows how much work remaining in the sprint. Maintained by scrum master daily.
- How much work completed/still pending?

Roles: the people who are involved.

Artefacts: what are the documents we have.

Ceremonies: what are the meetings we conduct.



Scrum Board: Status of the product in sprint. (TO DO -> IN progress -> Done)

Task: A task in testing is a specific activity that helps ensure a product or software meets its requirements and works as intended

Developer Tasks	QA Tasks
Under standing Requirements	Under standing Requirements
Desing	Writing Test Scenarios
Coding	Writing Test Cases
Unit Testing	Test Case Reviews
Integration Testing	Test Data Preparation
Code Review	Test Environment Setup
Bug Fixes	Test Execution
Team Meetings	Re-Testing Bugs
Any other...	Team Meetings
	Automation
	Any other...

Definition of Ready (DoR) & Definition of Done (DoD)

Definition of Ready (DoR)

- User Story is clear
- User Story is testable
- User Story is feasible
- User Story defined
- User Story Acceptance Criteria defined
- User Story dependencies identified
- User Story sized by Development Team
- Scrum Team accepts User Experience artefacts
- Performance criteria identified, where appropriate
- Team has a good idea what it will mean to Demo the User Story

Definition of Done (DoD)

- Code produced (all 'to do' items in code completed)
- Code commented, checked in and run against current version in source control
- Peer reviewed (or produced with pair programming) and meeting development standards
- Builds without errors
- Unit tests written and passing
- Deployed to system test environment and passed system tests
- Passed UAT (User Acceptance Testing) and signed off as meeting requirements
- Any build / deployment / configuration changes are implemented / documented / communicated
- Relevant documentation / diagrams produced and / or updated
- Remaining hours for task set to zero and task closed

Jira:

- It's a agile management tool / agile project management tool.

Different types of tools:

Test management, bug tracking, performance testing, automation, security testing tools.

Test Management Tools:

- Whatever activities are doing throughout testing process, organising the tasks like ts, tc and te.
- Each and every activity we can track in this tool.

Agile Tools:

- These tools are designed for automating the agile process.
 - Scrum related activities also do in the agile tools.
 - Jira, VersionOne, TeamCity...
-
- Jira is work on cloud instance.
 - Test cycle: Period of time for executing a set of manual tests.
 - Integration: separate people/things are brought together.
 - Every object is considering as issue in Jira.
 - Backlog contain epic along with user stories.

Jira project: (Agile/scrum activities)

- How to create project in Jira?
- How to add users/people in Jira?
- How to create backlog? (epic, user story)
- How to create stories & add story points.
- How to create sprint in Jira.
- Adding user stories to Jira.
- Starting sprint.
- Adding subtask to story.
- Sprint life cycle (task move through storyboard)
- How to install zephyr plugin?
- Test management activities (By default Jira will not provide these activities, we have to install special plugin called zephyr) (za fire)
 - Creating test cycles
 - Write testcases / import testcases to test cycle.
 - Execute the test cases.
 - Update Test cases (pass/fail/block)
 - Report bugs
 - Reports
 - Traceability matrix